

zeros

Create array of all zeros

Syntax

```
X = zeros
X = zeros(n)
X = zeros(sz1,...,szN)
X = zeros(sz)

X = zeros(__,typename)
X = zeros(__,like=p)
```

Description

X = zeros returns the scalar 0.

X = zeros(**n**) returns an n-by-n matrix of zeros. [example](#)

X = zeros(**sz1**,...,**szN**) returns an sz1-by-...-by-szN array of zeros where sz1,...,szN indicate the size of each dimension. For example, zeros(2,3) returns a 2-by-3 matrix. [example](#)

X = zeros(**sz**) returns an array of zeros where size vector sz defines size(X). For example, zeros([2 3]) returns a 2-by-3 matrix. [example](#)

X = zeros(__,**typename**) returns an array of zeros of data type typename. For example, zeros("int8") returns a scalar, 8-bit integer 0. You can use any of the input arguments in the previous syntaxes. [example](#)

X = zeros(__,**like=p**) returns an array of zeros like p; that is, of the same data type (class), sparsity, and complexity (real or complex) as p. You can specify typename or like, but not both. [example](#)

Examples

[collapse all](#)

Matrix of Zeros

Create a 4-by-4 matrix of zeros.

[Open in MATLAB Online](#)[Copy Command](#)

```
X = zeros(4)
```

[Get](#) ▼

```
X = 4x4
```

```
0     0     0     0
0     0     0     0
0     0     0     0
```

```
0    0    0    0
```

3-D Array of Zeros

Create a 2-by-3-by-4 array of zeros.

[Open in MATLAB Online](#)[Copy Command](#)

```
X = zeros(2,3,4);  
size(X)
```

[Get](#) ▼

```
ans = 1×3
```

```
2    3    4
```

Clone Size from Existing Array

Create an array of zeros that is the same size as an existing array.

[Open in MATLAB Online](#)[Copy Command](#)

```
A = [1 4; 2 5; 3 6];  
sz = size(A);  
X = zeros(sz)
```

[Get](#) ▼

```
X = 3×2
```

```
0    0  
0    0  
0    0
```

It is a common pattern to combine the previous two lines of code into a single line:

```
X = zeros(size(A));
```

[Get](#) ▼

Specify Data Type of Zeros

Create a 1-by-3 vector of zeros whose elements are 32-bit unsigned integers.

[Open in MATLAB Online](#)

 Copy Command

```
X = zeros(1,3,"uint32")
```

 Get ▾

```
X = 1x3 uint32 row vector
```

```
0    0    0
```

```
class(X)
```

 Get ▾

```
ans =  
'uint32'
```

Clone Complexity from Existing Array

Create a scalar 0 that is complex like an existing array instead of real valued.

First, create a complex vector.

[Open in MATLAB Online](#) Copy Command

```
p = [1+2i 3i];
```

 Get ▾

Create a scalar 0 that is complex like p.

```
X = zeros(like=p)
```

 Get ▾

```
X =  
0.0000 + 0.0000i
```

Clone Sparsity from Existing Array

Create a 10-by-10 sparse matrix.

[Open in MATLAB Online](#) Copy Command

```
p = sparse(10,10,pi);
```

 Get ▾

Create a 2-by-3 matrix of zeros that is sparse like p.

```
X = zeros(2,3,like=p)
```

 Get ▾

```
X = 2x3 sparse double matrix  
All zero
```

Clone Size and Data Type from Existing Array

Create a 2-by-3 array of 8-bit unsigned integers.

[Open in MATLAB Online](#)

[Copy Command](#)

```
p = uint8([1 3 5; 2 4 6]);
```

[Get](#) ▼

Create an array of zeros that is the same size and data type as p.

```
X = zeros(size(p),like=p)
```

[Get](#) ▼

```
X = 2x3 uint8 matrix
```

```
0  0  0
0  0  0
```

```
class(X)
```

[Get](#) ▼

```
ans =
'uint8'
```

Clone Distributed Array

If you have Parallel Computing Toolbox™, you can create a distributed array of zeros with an underlying type.

For example, create a 1000-by-1000 distributed array of zeros with underlying data type `int8`. Specify the underlying data type using the second argument and the primary data type using the third argument.

```
p = zeros(1000,"int8","distributed");
```

```
Starting parallel pool (parpool) using the 'local' profile ...
connected to 6 workers.
```

For the distributed data type, the `like` syntax of the `zeros` function clones the underlying data type in addition to the primary data type. Create an array of zeros that is the same size, primary data type, and underlying data type as p.

```
X = zeros(size(p),like=p);
class(X)
```

```
ans =
```

```
'distributed'
```

```
underlyingType(X)
```

```
ans =
```

```
'int8'
```

Input Arguments

[collapse all](#)

n — Size of square matrix
integer value

Size of square matrix, specified as an integer value.

- If n is 0, then X is an empty matrix.
- If n is negative, then it is treated as 0.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

sz1, ..., szN — Size of each dimension (as separate arguments)
integer values

Size of each dimension, specified as separate arguments of integer values.

- If the size of any dimension is 0, then X is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- Beyond the second dimension, zeros ignores trailing dimensions with a size of 1. For example, `zeros(3,1,1,1)` produces a 3-by-1 vector of zeros.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

sz — Size of each dimension (as a row vector)
integer values

Size of each dimension, specified as a row vector of integer values. Each element of this vector indicates the size of the corresponding dimension:

- If the size of any dimension is 0, then X is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- Beyond the second dimension, zeros ignores trailing dimensions with a size of 1. For example, `zeros([3 1 1])` produces a 3-by-1 vector of zeros.

Example: `sz = [2 3 4]` creates a 2-by-3-by-4 array.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

typename — Data type (class) to create
"double" (default) | "single" | "logical" | "int8" | "uint8" | ...

Data type (class) to create, specified as "double", "single", "logical", "int8", "uint8", "int16", "uint16", "int32", "uint32", "int64", "uint64", or the name of another class that provides zeros support.

p — Prototype of array to create
array

Prototype of array to create, specified as an array.

Data Types: double | single | logical | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

Complex Number Support: Yes

Output Arguments

[collapse all](#)

X — Array of zeros

scalar | vector | matrix | multidimensional array

Array of zeros, returned as a scalar, vector, matrix, or multidimensional array.

Extended Capabilities

[expand all](#)

C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

GPU Code Generation

Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

HDL Code Generation

Generate VHDL, Verilog and SystemVerilog code for FPGA and ASIC designs using HDL Coder™.

Thread-Based Environment

Run code in the background using MATLAB® backgroundPool or accelerate code with Parallel Computing Toolbox™ ThreadPool.

GPU Arrays

Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

Distributed Arrays

Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

Version History

Introduced before R2006a

See Also

[eye](#) | [ones](#) | [rand](#) | [createArray](#) | [complex](#) | [false](#) | [size](#)

Topics

[Class Support for Array-Creation Functions](#)

[Preallocation](#)

YOU MIGHT ALSO BE INTERESTED IN

